

FIT3003 Business intelligence and data warehousing - Assignment 2

Coen Pasitchnyj - 32528604

Preparation Stage: Exploration and Data Cleaning

I started by copying all the tables to my account and then getting the row count for every table to get an idea of how much data I am dealing with (and also because it might come in handy later):

```
SELECT 'BOOKING' TABLE_NAME, COUNT(*) ROW_COUNT FROM BOOKING
UNION ALL
SELECT 'MAINTENANCE', COUNT(*) FROM MAINTENANCE
UNION ALL
SELECT 'ACCIDENTINFO', COUNT(*) FROM ACCIDENTINFO
UNION ALL
SELECT 'CARACCIDENT', COUNT(*) FROM CARACCIDENT
UNION ALL
SELECT 'PASSENGER', COUNT(*) FROM PASSENGER
UNION ALL
SELECT 'CAR', COUNT(*) FROM CAR
UNION ALL
SELECT 'FACULTY', COUNT(*) FROM FACULTY
UNION ALL
SELECT 'MAINTENANCETEAM', COUNT(*) FROM MAINTENANCETEAM
UNION ALL
SELECT 'BELONGTO', COUNT(*) FROM BELONGTO
UNION ALL
SELECT 'RESEARCHCENTER', COUNT(*) FROM RESEARCHCENTER
UNION ALL
SELECT 'ERROR', COUNT(*) FROM ERROR
UNION ALL
SELECT 'MAINTENANCETYPE', COUNT(*) FROM MAINTENANCETYPE;
```

	TABLE_NAME	ROW_COUNT
1	BOOKING	10001
2	MAINTENANCE	1001
3	ACCIDENTINFO	994
4	CARACCIDENT	1000
5	PASSENGER	154
6	CAR	30
7	FACULTY	5
8	MAINTENANCETEAM	5
9	BELONGTO	12
10	RESEARCHCENTER	4
11	ERROR	5
12	MAINTENANCETYPE	5

Checking for data duplication between records

Next I checked if there were any duplicate primary keys in any record in any of the tables. The command looks like so using the AccidentInfo table as an example:

```
SELECT ACCIDENTID, COUNT(*)
FROM ACCIDENTINFO
GROUP BY ACCIDENTID
HAVING COUNT(*) > 1;
```

I repeated this command for all the tables in the operational database. All returned 0 except for the Booking table:

```
SELECT BOOKINGID, COUNT(*)
FROM BOOKING
GROUP BY BOOKINGID
HAVING COUNT(*) > 1;
```

	BOOKINGID	COUNT(*)
1	T1218	2

Looking at the specific booking ID:

```
SELECT *
FROM BOOKING
WHERE BOOKINGID = 'T1218';
```

	BOOKINGID	REGISTRATIONNO	BOOKINGDATE	DEPARTUREZONE	DESTINATIONZONE	PASSENGERID
1	T1218	Car14	29/JUL/18	ZoneD	ZoneB	U059
2	T1218	Car14	29/JUL/18	ZoneD	ZoneB	U059

And then looking specifically at the date and time:

```
SELECT to_char(BOOKINGDATE, 'YYYY-MM-DD HH24:MI:SS') as BOOKINGDATE
FROM BOOKING
WHERE BOOKINGID = 'T1218';
```

	BOOKINGDATE
1	2018-07-29 00:00:00
2	2018-07-29 00:00:00

We can see the entire row has been exactly duplicated, so we can remove one of the entries safely. I could do this by running a “create table as select distinct” statement, which would make a new table, but to avoid disrupting any dependent tables or primary/foreign key rules I decided to just delete the row in place:

```
DELETE FROM BOOKING
WHERE ROWID NOT IN (
    SELECT MIN(ROWID)
    FROM BOOKING
    GROUP BY BOOKINGID
);
```

Then checking that there is now only one booking with ID T1218:

All rows fetched: 1 in 0.021 seconds

	BOOKINGID	REGISTRATIONNO	BOOKINGDATE	DEPARTUREZONE	DESTINATIONZONE	PASSENGERID
1	T1218	Car14	29/JUL/18	ZoneD	ZoneB	U059

So the duplicate was successfully removed.

Another potential duplicate I found was in the Maintenance table. There are quite a few examples of a single car having two maintenances in one day, but usually they are different maintenance types. There is one example of a single car having 2 of the same maintenance on the same day, with the same cost and performed by the same team, which I found with this query:

```
SELECT REGISTRATIONNO, MAINTENANCEDATE, MAINTENANCETYPE, COUNT(*)
FROM MAINTENANCE
GROUP BY REGISTRATIONNO, MAINTENANCEDATE, MAINTENANCETYPE
HAVING COUNT(*) > 1
ORDER BY REGISTRATIONNO, MAINTENANCEDATE, MAINTENANCETYPE;
```

All rows fetched: 1 in 0.020 seconds

	REGISTRATIONNO	MAINTENANCEDATE	MAINTENANCETYPE	COUNT(*)
1	Car13	19/JUL/15	M002	2

Looking closer, this happens to be one of the entries that I already have fixed up (it had negative maintenance cost, this is shown later in the report):

```
SELECT MAINTENANCEID, REGISTRATIONNO, MAINTENANCEDATE,
TO_CHAR(MAINTENANCEDATE, 'HH24:MI') AS MAINTENANCETIME,
MAINTENANCETYPE, MAINTENANCECOST, TEAMID
FROM MAINTENANCE
WHERE REGISTRATIONNO = 'Car13'
AND MAINTENANCEDATE = TO_DATE('19/JUL/2015', 'DD/MON/YYYY');
```

All rows fetched: 2 in 0.024 seconds

	MAINTENANCEID	REGISTRATIONNO	MAINTENANCEDATE	MAINTENANCETIME	MAINTENANCETYPE	MAINTENANCECOST	TEAMID
1	M936	Car13	19/JUL/15	00:00	M002	200	T004
2	M2000	Car13	19/JUL/15	00:00	M002	200	T004

So the question here is: is this a duplicated record, and should I remove it? It is certainly suspicious, however MaintenanceType M002 corresponds to “Hardware Upgrade:Sensor”. It is not unreasonable for a car to have two sensors upgraded in a day, and because the MaintenanceDate does not store the time of the maintenance, I cannot say that this record is duplicated and it may well be two separate maintenances, so I decided to leave it.

Checking for invalid foreign key values

Next I checked for FK values that are invalid (do not exist in the table they are supposed to be linking to).

The textbook shows that we can do this by using this query:

```
SELECT *
FROM <<TABLE 1>>
WHERE <<FK COLUMN>> NOT IN (
    SELECT <<PK COLUMN>>
    FROM <<TABLE 2>>
);
```

I identified two problems, one in the Passenger table:

```
SELECT *
FROM PASSENGER
WHERE FACULTYID NOT IN (
    SELECT FACULTYID
    FROM FACULTY
);
```

All rows fetched: 1 in 0.018 seconds

	PASSENGERID	PASSENGERNAME	PASSENGERROLE	PASSENGERGENDER	PASSENGERAGE	FACULTYID
1	U163	Anabia Mccabe	Staff	Male	21	Alienware

The FacultyID has been entered as “Alienware”, which is certainly not a FacultyID.

The other problem is in the AccidentInfo table:

```
SELECT *
FROM ACCIDENTINFO
WHERE ERRORCODE NOT IN (
    SELECT ERRORCODE
    FROM ERROR
);
```

All rows fetched: 1 in 0.021 seconds

	ACCIDENTID	ACCIDENTZONE	CAR_DAMAGE_SEVERITY	ERRORCODE
1	A2000	ZoneB	No damage	Error010

Here the error code is not obviously wrong visually, but it does not exist in the Error table:

All rows fetched: 5 in 0.020 seconds

	ERRORCODE	ERRORMESSAGE
1	Error001	Image recognition system failed
2	Error002	Low Battery
3	Error003	Signal loss: front sensor
4	Error004	Lidar systeem failed:Unable to locate
5	Error005	Unknow issue: please contact with maintenance team

*Also, there are some spelling mistakes and incorrect grammar in the error messages, but they are still understandable and it does not mean the data is “wrong” so I will leave them.

For the invalid PassengerID, I checked if that passenger made any bookings and discovered that they had not:

```
SELECT *
FROM BOOKING
WHERE PASSENGERID = 'U163';
```

All rows fetched: 0 in 0.030 seconds

	BOOKINGID	REGISTRATIONNO	BOOKINGDATE	DEPARTUREZONE	DESTINATIONZONE	PASSENGERID

The textbook says the easiest thing to do would be to set the invalid FK value to NULL, however there is a NOT NULL constraint on this FK.

Because I can’t just choose a Faculty for the passenger, and because they have no bookings anyway, I decided to simply delete the passenger for the purpose of creating the warehouse as it should not affect the fact measures (the business is not interested in asking about the raw number of registered passengers).

```
DELETE FROM PASSENGER
WHERE PASSENGERID = 'U163';
```

Now we can see there are no more passengers with invalid FacultyID’s:

```
SELECT *
FROM PASSENGER
WHERE FACULTYID NOT IN (
    SELECT FACULTYID
    FROM FACULTY
);
```

All rows fetched: 0 in 0.019 seconds

	PASSENGERID	PASSENGERNAME	PASSENGERROLE	PASSENGERGENDER	PASSENGERAGE	FACULTYID

For the invalid ErrorCode in the AccidentInfo table, because it is so close to one of the valid codes, I will infer that “Error010” was an accidental typo when entering “Error001”, and I will amend it to match this value:

```
UPDATE ACCIDENTINFO
SET ERRORCODE = 'Error001'
WHERE ACCIDENTID = 'A2000' AND ERRORCODE = 'Error010';
```

We can see the row is updated now:

```
SELECT *
FROM ACCIDENTINFO
WHERE ACCIDENTID = 'A2000';
```

All rows fetched: 1 in 0.015 seconds

	ACCIDENTID	ACCIDENTZONE	CAR_DAMAGE_SEVERITY	ERRORCODE
1	A2000	ZoneB	No damage	Error001

Checking for NULLs in the tables

Now I want to scan for any NULLs in the database. For the small tables I just visually scanned with a SELECT *, and for the larger ones I ran queries like this for each:

```
SELECT *
FROM ACCIDENTINFO
WHERE ACCIDENTID IS NULL
OR ACCIDENTZONE IS NULL
OR CAR_DAMAGE_SEVERITY IS NULL
OR ERRORCODE IS NULL;
```

I only found one NULL, which was in the AccidentInfo table query shown above:

All rows fetched: 1 in 0.018 seconds

	ACCIDENTID	ACCIDENTZONE	CAR_DAMAGE_SEVERITY	ERRORCODE
1	(null)	ZoneC	Severe damage	Error005

The primary key AccidentID is missing, so this will be quite tricky to fix, and crucial because this connects to the bridge table CarAccident and links cars to accidents, so it is important for our fact measures.

First I want to check if there are any CarAccident rows with NULLs or with an AccidentID that does not exist in the AccidentInfo table.

- If there are some with a NULL, I could create a new distinct AccidentID and link them (but this is assuming that they belong together).

- If there are rows with an AccidentID that doesn't exist in AccidentInfo, I could infer that that is the missing ID and restore the PK.

Unfortunately there are no NULL AccidentIDs in CarAccident:

```
SELECT *
FROM CARACCIDENT
WHERE ACCIDENTID IS NULL;
```

All rows fetched: 0 in 0.015 seconds

	REGISTRATIONNO	ACCIDENTID

There are also no CarAccident rows with an AccidentID that does not exist in the AccidentInfo table:

```
SELECT *
FROM CARACCIDENT
WHERE ACCIDENTID NOT IN (
    SELECT ACCIDENTID
    FROM ACCIDENTINFO
);
```

All rows fetched: 0 in 0.020 seconds

	REGISTRATIONNO	ACCIDENTID

So therefore I am not able to restore the relationship here. I decided to check if it is maybe a duplicate row that just happened to be missing its ID by checking other AccidentInfo rows with the same other fields:

```
SELECT *
FROM ACCIDENTINFO
WHERE ACCIDENTZONE = 'ZoneC'
AND CAR_DAMAGE_SEVERITY = 'Severe damage'
AND ERRORCODE = 'Error005';
```

All rows fetched: 16 in 0.020 seconds

	ACCIDENTID	ACCIDENTZONE	CAR_DAMAGE_SEVERITY	ERRORCODE
1	A636	ZoneC	Severe damage	Error005
2	A660	ZoneC	Severe damage	Error005
3	A721	ZoneC	Severe damage	Error005
4	A365	ZoneC	Severe damage	Error005
5	A403	ZoneC	Severe damage	Error005
6	A496	ZoneC	Severe damage	Error005
7	A514	ZoneC	Severe damage	Error005
8	A545	ZoneC	Severe damage	Error005
9	A580	ZoneC	Severe damage	Error005
10	A978	ZoneC	Severe damage	Error005
11	A1002	ZoneC	Severe damage	Error005
12	A1159	ZoneC	Severe damage	Error005
13	A1240	ZoneC	Severe damage	Error005
14	A895	ZoneC	Severe damage	Error005
15	A897	ZoneC	Severe damage	Error005
16	(null)	ZoneC	Severe damage	Error005

However there are 15 other matching records, so I couldn't say that it is a duplicate of any specific one of them.

So, I am unable to restore the relationship, I cannot say it is a duplicate, and there are no CarAccident rows that are not linked to a valid other AccidentInfo row. The remaining attributes of the NULL AccidentID row seem valid, but they are not unique.

Because all accidents in this scenario should be associated with a car, and this one cannot be, it cannot be reliably used in the accidents analysis. Therefore I arrived at the conclusion that I should treat the entry as a mistake and remove it from the database. This will of course change the total accidents fact measure but the existence of this row seems too anomalous to ignore. Doing so will also not leave any unconnected CarAccident bridge table rows, so it should not cause problems there when joining later.

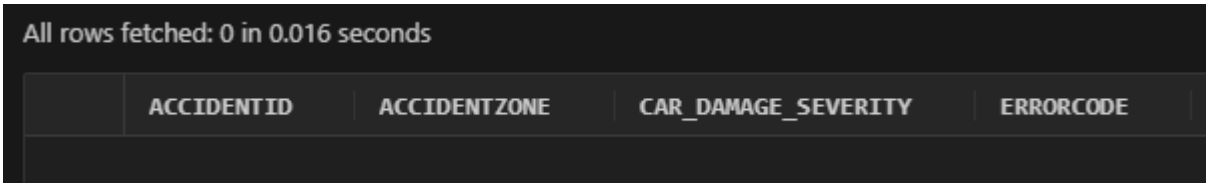
Deleting the AccidentInfo row with NULL AccidentID (I can do it simply like this because I know there is only one row affected:

```
DELETE FROM ACCIDENTINFO
WHERE ACCIDENTID IS NULL;
```

Then re-running the query from before to verify there are no more NULLs in the AccidentInfo table:

```
SELECT *
FROM ACCIDENTINFO
```

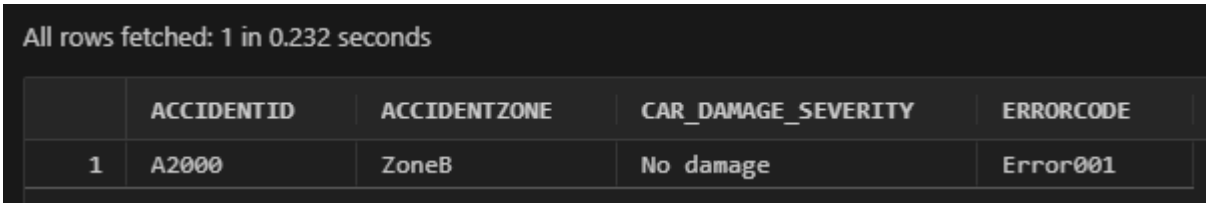
```
WHERE ACCIDENTID IS NULL
OR ACCIDENTZONE IS NULL
OR CAR_DAMAGE_SEVERITY IS NULL
OR ERRORCODE IS NULL;
```



CarAccident -> AccidentInfo relationship problem

While fixing the previous issue, because of my assumption that an Accident should involve at least one car in this scenario, I also ran a query to check if there were any other Accidents that were not actually linked to a car:

```
SELECT *
FROM ACCIDENTINFO
WHERE ACCIDENTID NOT IN (
    SELECT ACCIDENTID
    FROM CARACCIDENT
);
```



I found one, and for the same reason as above (it does not fit with the valid assumptions about the scenario, i.e how can an accident exist that did not involve a car?), and because assigning a car to it would be unsupported, I decided to just remove it from the database so that it would not interfere with the facts:

```
DELETE FROM ACCIDENTINFO
WHERE ACCIDENTID = 'A2000';
```

And then checking that it has been removed:

```
SELECT *
FROM ACCIDENTINFO
WHERE ACCIDENTID = 'A2000';
```

All rows fetched: 0 in 0.017 seconds

	ACCIDENTID	ACCIDENTZONE	CAR_DAMAGE_SEVERITY	ERRORCODE

So now all Accidents are valid and involve at least one car.

Checking for other relationship issues

After finding the above example, I decided to look for any other relationship problems.

First I checked that all maintenance teams belong to a centre, and this was all good:

```
SELECT *
FROM MAINTENANCETEAM
WHERE TEAMID NOT IN (
    SELECT TEAMID
    FROM BELONGTO
);
```

All rows fetched: 0 in 0.024 seconds

	TEAMID	TEAMLEADER	PHONE

I checked a few others, such as if there are any passengers with no bookings (but this would be fine anyway as a passenger would be entered into the database as part of a registration process before they can book) and found no other issues of this kind.

Checking for other invalid or incorrectly entered values

The other incorrect data I found was a negative MaintenanceCost:

```
SELECT *
FROM MAINTENANCE
WHERE MAINTENANCECOST < 0;
```

All rows fetched: 1 in 0.015 seconds

	MAINTENANCEID	REGISTRATIONNO	MAINTENANCEDATE	MAINTENANCETYPE	MAINTENANCECOST	TEAMID
1	M2000	Car13	19/JUL/15	M002	-200	T004

I had a look at the other maintenance costs:

```
SELECT MAINTENANCECOST, COUNT(*)
FROM MAINTENANCE
GROUP BY MAINTENANCECOST
ORDER BY MAINTENANCECOST;
```

All rows fetched: 6 in 0.015 seconds

	MAINTENANCECOST	COUNT(*)
1	-200	1
2	100	204
3	200	197
4	300	197
5	400	172
6	500	230

From looking at the other common costs, it is quite reasonable to assume that the -200 value was probably a typo when trying to enter “200”, so in this case I will adjust it to that rather than removing the record:

```
UPDATE MAINTENANCE
SET MAINTENANCECOST = 200
WHERE MAINTENANCEID = 'M2000';
```

All rows fetched: 1 in 0.018 seconds

	MAINTENANCEID	REGISTRATIONNO	MAINTENANCEDATE	MAINTENANCETYPE	MAINTENANCECOST	TEAMID
1	M2000	Car13	19/JUL/15	M002	200	T004

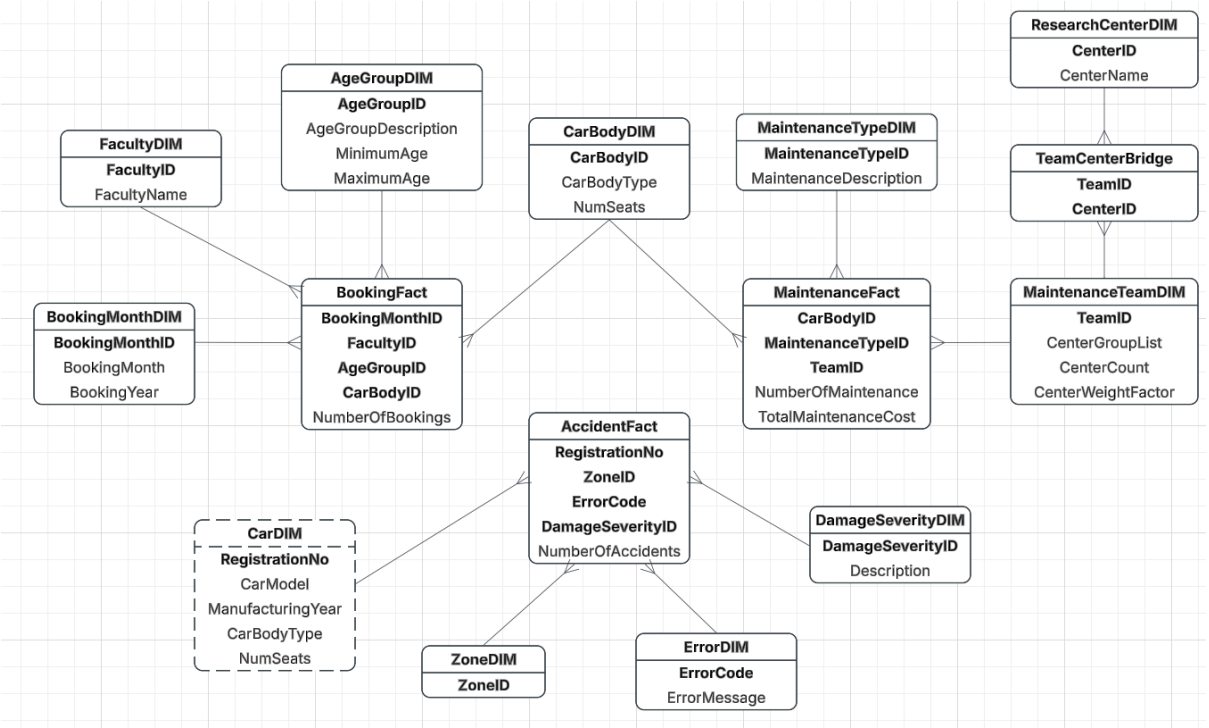
Overall

So, overall I was able to find and fix 6 data anomalies:

- 1 duplicate row
- 2 invalid foreign keys
- 1 NULL primary key
- 1 relationship problem
- 1 invalid/unreasonable value

Version 1 Snowflake Schema

Pictured here is my version 1 snowflake schema, which is at the highest level of aggregation and holds the minimum detail required to answer the provided business queries.



The four fact measures that the business asks about are:

- Number of booking records
- Number of maintenance records
- Total maintenance cost
- Number of accident records

A few examples of some dimensions from the business queries are:

- Passenger age group
- Faculty
- Research centre
- Month
- Vehicle/ vehicle type
- Maintenance type
- Error code
- Damage severity

We can use a few of these dimensions to apply the two column table test. Passenger age group makes sense for booking counts, but not for maintenance cost. Error code makes sense for accident counts, but not for booking counts. Research centre makes sense for filtering the maintenance fact measures, but not accident counts.

Therefore, we have 3 separate subjects (BookingFact, MaintenanceFact, and AccidentFact), and the schema needs 3 separate fact tables.

BookingFact Design

The assignment questions regarding bookings centre around 4 dimensions:

- The passenger's faculty
- The passenger's age group
- The body type of the vehicle being booked
- The month of the booking

So I have linked the BookingFact to 4 dimensions representing those 4 dot points, and the only measure is the number of bookings.

I want to briefly explain one design choice here, which was the use of the CarBodyDIM, rather than linking the BookingFact to the CarDIM. This is because the assignment does not ask for bookings per car in any way, and linking the BookingFact to the CarDIM would allow us to do that. Version 1 of the star schema is supposed to be at the highest level of aggregation, and doing so would actually make our schema more granular than it needs to be. Hence I decided to create a new dimension that only stores the body type of the vehicle being booked.

MaintenanceFact Design

The assignment questions regarding maintenances centre around 3 dimensions:

- The body type of the vehicle undergoing maintenance
- The type of the maintenance
- The research centre of the team performing the maintenance.

There are two measures being asked for in the assignment questions: number of maintenances, and also the total maintenance cost.

The MaintenanceFact is linked to the CarBodyDIM for the same reason as the BookingFact (linking to the CarDIM would allow us to filter by body type, but would be more granular than required to answer the questions).

A bridge table is needed to link to the research centre, because one maintenance record has one maintenance team, but that one maintenance team can belong to many research centres. Also, one research centre can be associated with many teams and many maintenance records.

The TeamCentreBridge table will essentially just be the BelongTo table from the operational database. I will store the aggregate centre list, the centre count, and the weight factor for each team in the MaintenanceTeamDIM rather than in the bridge table, because otherwise I will just be duplicating the data a lot.

This is probably technically not the absolute highest level of aggregation because this design allows us to filter maintenances by the team, but it is the most natural and easy to use solution for the many to many relationship, and is what was shown in the textbook.

AccidentFact Design

The assignment questions regarding accidents centre around 4 dimensions:

- The specific car/s involved
- The zone in which the accident occurred
- The error code for the accident
- The severity of the damage.

So it is quite simple to design those 4 dimensions and connect them to the AccidentFact. The questions only ask about the number of accidents, so that is the only fact measure.

However, because one accident can involve multiple cars, that accident can contribute one count to Car01 and one count to Car02 for example, and therefore we might overestimate the number of unique accidents that occurred.

Therefore CarDIM is a determinant dimension for the AccidentFact in my version 1 design. The car must be included in the query for the measure to have useful meaning. This is valid, as all the questions involving accidents specify an exact car.

Increasing V1's Granularity

Here I will detail the changes I would make to the version 1 star schema in order to increase its granularity to level 0 (no aggregation).This means that in version 2, each fact row needs to represent one operational transaction with no grouping (and count measures should be 1).

BookingFact Changes

- Replace BookingMonthID with the exact BookingDate and BookingTime, and split the BookingMonthDIM into BookingDateDIM and BookingTimeDIM.
- Replace FacultyID and AgeGroupID with the specific PassengerID. This will give us individual passenger details. Also introduce a PassengerDIM of course (the AgeGroupDIM could remain branched to the PassengerDIM, or we could fold it into the PassengerDIM to allow for easy filtering).
- Remove CarBodyDIM, and instead link BookingFact to CarDIM through RegistrationNo instead. CarBodyType and NumSeats can still be obtained from CarDIM.
- I would also include the original BookingID as a dimensionless key, just to absolutely ensure that every row in the fact table is unique and at the highest granularity.

MaintenanceFact Changes

- Similar to BookingFact, replace the link to CarBodyDIM with a link to CarDIM so each record is specific to one car (change CarBodyID with RegistrationNo(.
- Add a MaintenanceDateDIM with primary key MaintenanceDate, and link it to the MaintenanceFact table. I looked into the provided database and a time is not recorded for maintenance, only the date.

- TeamID is already included, and there is one TeamID per maintenance record, so this will remain as well as the bridge to the research centre.
- Similar to the BookingFact changes, I would add the MaintenanceID (as a dimensionless key) from the database to ensure every row is unique. During exploration, I discovered and provided an example of two identical maintenance records for the same car at the same time, but reasoned that it might be two separate sensors being upgraded but recorded as separate maintenances for business reasons.

AccidentFact Changes

- For the accident fact, the zone, error code and damage severity cannot be lowered any more, so they remain as they are.
- The easiest way to ensure every fact row refers to one event is to include the AccidentID from the database. A car cannot be involved twice in one accident, so this ensures maximum granularity.
    - However, this means that CarDIM remains as a determinant dimension for the same reason as in the version 1 schema.
    - I could introduce a car-accident bridge table, but this is more complicated and is not needed to answer the assignment questions (as they always specify a car).

SCD Types and Temporal Dimension Example

SCD Type 0
Type 0 permanently stores the original value of the attribute. No history is recorded, and reports always show the first value loaded into the warehouse.

SCD Type 1
Type 1 records whatever the latest value of the attribute is, and overwrites the previous one when needed. No history is recorded, and reports will always show the latest value.

SCD Type 2
A new dimension row is inserted for every change to the attribute. Sometimes a sequence number might be appended to the PK to keep track of changes, and a flag to track the current value can also be used. The full history is recorded but stays within the dimension table. Reports can single out the valid value for the time at which the event occurred.

SCD Type 3
Type 3 is a simplification of type 2. It keeps a limited history, only the current and previous values as separate columns in each row (assumes earlier values are not needed). The record of the SCD remains in the dimension table.

SCD Type 4
In type 4, the entire history of the time dependent attribute is split into a new history dimension table, and the attribute is removed from the original dimension. In the new table,

all values are recorded, along with their start date and end date, and the dimensions are linked using the PK from the original table. The historical value is selected using the fact date.

SCD Type 6
Type 6 is a combination between type 2 and 3. It records the complete history of the changing attribute in the dimension itself, while also recording the current and previous value. It doesn't require a separate identifier for each row, because the PK is expanded to contain the original PK, as well as the start and end date for the changing attribute. So it avoids having a separate dimension, but makes joins more complicated because of the expanded PK.

Temporal Dimension Example

In my version 2 star schema, I have included a PassengerDIM table, which records things such as the passenger's age (as well as age group for easy filtering), their faculty, their role etc. Realistically, these things may change with time: the passenger might change faculty or change from a student to staff when they graduate for example.

I believe it would be wise to include some kind of temporal history for these passengers, especially because the business wishes to filter some of the fact by the passenger's faculty or age group, and we should make sure that it uses the faculty/age group the passenger belonged to at the time of the booking (in case we are looking at older bookings).

I would propose using a type 2 SCD design, and add a surrogate key PassengerKey (while also keeping the PassengerID from the database). Each time a passenger changes role/faculty/age group, we could create a new line in the database with a new PassengerKey but the same PassengerID, and the updated values. There would be a CurrentFlag column to tell us which is the current state of that passenger. There would also be a start and end date column in each entry, and the end date for the current entry could simply be Dec/31/9999.

The BookingFact would then reference the PassengerKey. Type 2 is an appropriate choice of SCD for PassengerDIM because:

- It will preserve the complete history of each passenger.
- It will allow for historically accurate reporting on passenger bookings when filtering by the changing attributes.
- It does not require a new PassengerHistory table, and allows for any number of changes.
- Type 2 does add more rows to the table, but the attributes of faculty, role, and age group are very slow changing. I.e a passenger would rarely change faculty, would probably only change role once ever, and rarely cross into a new age group. So the number of rows will not expand massively.

Type 0, 1 and 3 do not record the full history, so cannot be used for correct historic reporting. Type 4 could work but involved making a whole new dimension, which is unnecessary work and adds complexity. Type 6 is very similar but adds the current and previous columns which

are also not strictly required. Type 2 is the simplest solution that satisfies our needs in this case.

Version 1 Implementation

Below are the screenshots of all the table structures I created as part of the implementation of my version 1 STAR schema.

BookingMonthDIM

	COLUMN_NAME	DATA_TYPE	NULLABLE	INVISIBLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	BOOKINGMONTHID	VARCHAR2(6 BYTE)	No	No	(null)	1	(null)
2	BOOKINGMONTH	VARCHAR2(2 BYTE)	Yes	No	(null)	2	(null)
3	BOOKINGYEAR	VARCHAR2(4 BYTE)	Yes	No	(null)	3	(null)

FacultyDIM

	COLUMN_NAME	DATA_TYPE	NULLABLE	INVISIBLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	FACULTYID	VARCHAR2(26 BYTE)	No	No	(null)	1	(null)
2	FACULTYNAME	VARCHAR2(256 BYTE)	Yes	No	(null)	2	(null)

AgeGroupDIM

	COLUMN_NAME	DATA_TYPE	NULLABLE	INVISIBLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	AGEGROUPID	NUMBER	No	No	(null)	1	(null)
2	AGEGROUPDESCRIPTION	VARCHAR2(50 BYTE)	No	No	(null)	2	(null)
3	MINIMUMAGE	NUMBER	No	No	(null)	3	(null)
4	MAXIMUMAGE	NUMBER	No	No	(null)	4	(null)

CarBodyDIM

	COLUMN_NAME	DATA_TYPE	NULLABLE	INVISIBLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	CARBODYID	NUMBER	No	No	(null)	1	(null)
2	CARBODYTYPE	VARCHAR2(26 BYTE)	Yes	No	(null)	2	(null)
3	NUMSEATS	NUMBER(38,0)	Yes	No	(null)	3	(null)

MaintenanceTypeDIM

	COLUMN_NAME	DATA_TYPE	NULLABLE	INVISIBLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	MAINTENANCETYPEID	VARCHAR2(26 BYTE)	No	No	(null)	1	(null)
2	MAINTENANCEDESCRIPTION	VARCHAR2(128 BYTE)	Yes	No	(null)	2	(null)

MaintenanceTeamDIM

	COLUMN_NAME	DATA_TYPE	NULLABLE	INVISIBLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	TEAMID	VARCHAR2(26 BYTE)	No	No	(null)	1	(null)
2	CENTERGROUPLIST	VARCHAR2(4000 BYTE)	Yes	No	(null)	2	(null)
3	CENTERCOUNT	NUMBER	Yes	No	(null)	3	(null)
4	CENTERWEIGHTFACTOR	NUMBER	Yes	No	(null)	4	(null)

TeamCenterBridge

	COLUMN_NAME	DATA_TYPE	NULLABLE	INVISIBLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	TEAMID	VARCHAR2(26 BYTE)	No	No	(null)	1	(null)
2	CENTERID	VARCHAR2(26 BYTE)	No	No	(null)	2	(null)

ResearchCenterDIM

	COLUMN_NAME	DATA_TYPE	NULLABLE	INVISIBLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	CENTERID	VARCHAR2(26 BYTE)	No	No	(null)	1	(null)
2	CENTERNAME	VARCHAR2(128 BYTE)	Yes	No	(null)	2	(null)

CarDIM

	COLUMN_NAME	DATA_TYPE	NULLABLE	INVISIBLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	REGISTRATIONNO	VARCHAR2(26 BYTE)	No	No	(null)	1	(null)
2	CARMODEL	VARCHAR2(26 BYTE)	Yes	No	(null)	2	(null)
3	MANUFACTURINGYEAR	NUMBER(38,0)	Yes	No	(null)	3	(null)
4	CARBODYTYPE	VARCHAR2(26 BYTE)	Yes	No	(null)	4	(null)
5	NUMSEATS	NUMBER(38,0)	Yes	No	(null)	5	(null)

ZoneDIM

	COLUMN_NAME	DATA_TYPE	NULLABLE	INVISIBLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	ZONEID	VARCHAR2(2…	No	No	(null)	1	(null)

ErrorDIM

	COLUMN_NAME	DATA_TYPE	NULLABLE	INVISIBLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	ERRORCODE	VARCHAR2(26 BYTE)	No	No	(null)	1	(null)
2	ERRORMESSAGE	VARCHAR2(128 BYTE)	Yes	No	(null)	2	(null)

DamageSeverityDIM

	COLUMN_NAME	DATA_TYPE	NULLABLE	INVISIBLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	DAMAGESEVERITYID	NUMBER	No	No	(null)	1	(null)
2	DESCRIPTION	VARCHAR2(50 BYTE)	No	No	(null)	2	(null)

Data Analysis

Findings Report

MonCity recorded 10,000 bookings, 1,001 maintenances costing $302,900 total, and 1,000 car-accident involvements. Demand was distributed almost evenly between the 3 types of vehicle. People movers were booked 3,396 times, mini buses 3,313, and buses 3,291. Customer demand alone does not point to one vehicle type to invest further in (Table 1).

Vehicle-type investment scorecard

Car Type	Fleet Vehicles	Bookings	Bookings/Vehicle	Maintenances	Maintenance Cost/Vehicle	Maintenance Cost/Booking	Accidents/Vehicle	Severe Damage %
Bus (40 seats)	10	3291	329.10	355	$11,350.00	$34.49	30.50	25.57%
Mini Bus (20 seats)	10	3313	331.30	312	$8,880.00	$26.80	32.40	29.94%
People Mover (10 seats)	10	3396	339.60	334	$10,060.00	$29.62	37.10	0.54%
Total	**30**	**10000**	**333.33**	**1001**	**$10,096.67**	**$30.29**	**33.33**	**17.70%**

Table 1

Maintenance and accident measures reveal bigger differences. Mini buses have the lowest maintenance cost at $8,880/vehicle and $26.80/booking. People movers rank second at $10,060/vehicle and $29.62/booking, and buses are the most expensive at $11,350/vehicle and $34.49/booking. Regular maintenance is the largest cost driver, accounting for $115,000, or 38% of total maintenance expenditure (Table 2). Regular maintenance had the most occurrences (230) and the highest cost ($500).

Maintenance Desc	Total Cost	Maintenances	Cost per Maintenance
Regular maintenance	$115,000.00	230	$500.00
Battery replacement	$68,800.00	172	$400.00
Hardware Upgrade:Lidar system	$59,100.00	197	$300.00
Hardware Upgrade:Sensor	$39,600.00	198	$200.00
System Upgrade	$20,400.00	204	$100.00
Total	**$302,900.00**	**1001**	**$302.60**

Table 2

People movers had the highest accident frequency (371 involvements), equivalent to 10.92/100 bookings. Mini Buses recorded 9.78 and Buses 9.27/100 bookings. However, the accident severity results greatly change how this is interpreted. Only two people mover accidents had severe damage (0.54% of its accident involvements). Mini buses recorded 97 severe accidents (29.94%) and buses recorded 78 (25.57%, Figure 1). Mini Buses therefore accounted for approximately 54.8% of all severe damage accidents despite receiving only 33.13% of bookings.

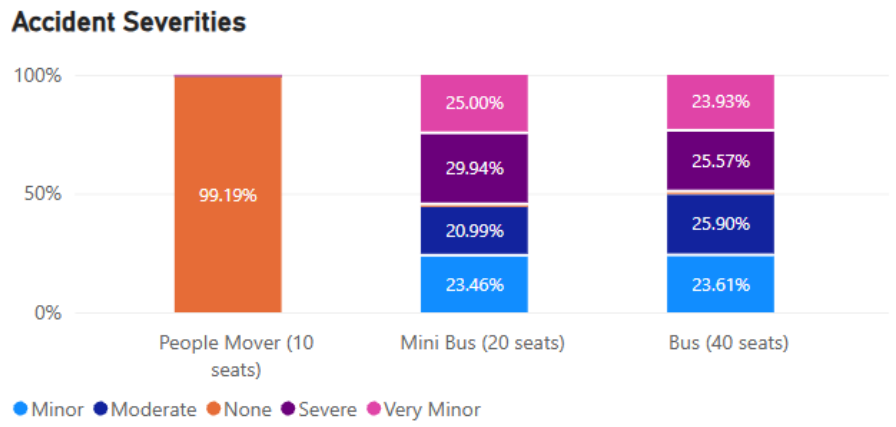


Figure 1

Of the 371 people mover accident involvements, 368 caused no damage, leaving only 3 damaging events, 2 of which were severe. In contrast, only 2 of 324 mini bus involvements and 3 of 305 bus involvements caused no damage. This extreme difference may represent safer incidents, but could also be a result of reporting or classification differences. MonCity should look into the cause of this difference before treating people movers as inherently safer.

The most common error code in accidents was Error002: Low battery, associated with 373 accidents (37.3% of all accident involvements). This was more than twice the 169 involvements from the next error (Error001). Battery replacement was also the second largest maintenance cost category, costing $68,800.

MonCity should prioritise further investment in people movers because they combine the highest booking demand, moderate maintenance/operating cost, and very low severe damage rates (despite having the most accidents in total). However, expansion of the people mover fleet should also hinge on reducing this high frequency of low severity accidents. Since the low battery error was the main error associated with accidents, some preventative maintenance on the fleet's batteries may help lower the amount of accidents. Additionally, annual bookings do not show sustained growth, so this expansion should be gradual Figure 2). Mini bus investment should be paused until the reason for their high rate of severe damage in accidents can be understood, as this represents high repair costs for the business and poor safety for travellers. Furthermore, MonCity should also look to reduce the costs of the regular maintenance (which is the main maintenance spend), potentially through better deals with the vehicle dealerships.

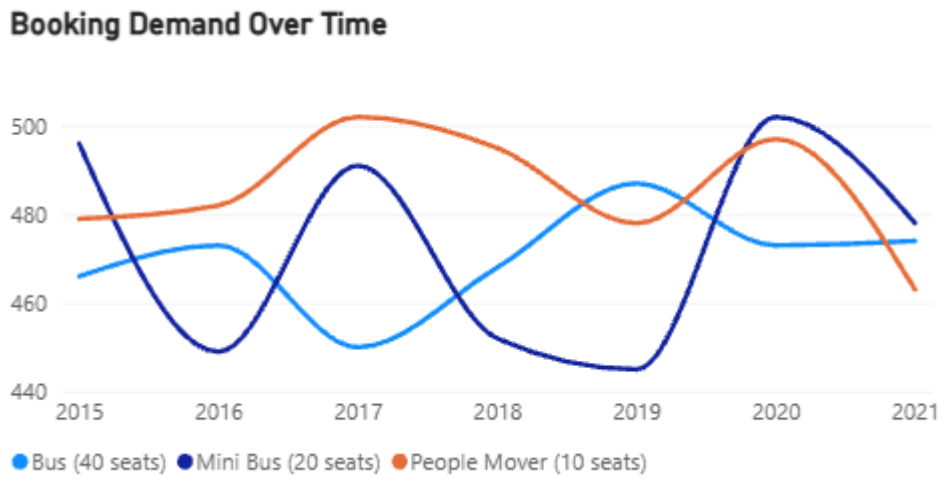


Figure 2